



Auditoria Centro Deportivo

Integrantes:

Bassi Servant, Tomas

Benedetti, Matias

Universidad Tecnológica Nacional

Metodología de sistemas II

Profesores:

Halm, Cristian

Martin, Lucrecia

Fecha:

20/11/2025

Índice

Índice	2
Descripción del dominio	4
Misión	4
Alcance	4
Descripción general	4
Listado de requisitos funcionales	5
Tecnologías	6
Backend	6
Frontend	6
Detalle de arquitectura	6
Nivel 1: Diagrama de contexto	6
Descripción	7
Responsabilidades	7
Nivel 2: Diagrama de contenedores	8
Nivel 3: Diagrama de componentes	9
Capa de Controllers	9
Capa de Services	9
Capa de Repository	10
Diagrama de casos de uso	10
Estado actual	11
PROPUESTA DE MEJORA 1: Violación del principio de responsabilidad unica (SRP) en EnrollmentService	11
Módulo/Paquete	11
Descripción del Dominio	11
Código problemático	11
Solución a implementar	12
Ventajas	12
Desventajas	12
Propuesta de mejora 2: Duplicación de código	12
Módulo/Paquete	12
Descripción del Dominio	12
Código problemático	13
Solución a implementar	13
Ventajas	14
Desventajas	14
Propuesta de mejora 3: Alto acoplamiento en MemberService	14
Módulo/Paquete	14
Descripción del Dominio	14
Código problemático	15
Solución a Implementar	15
Ventajas	15
Desventajas	16



Propuesta de mejora 4: Patrón factory para creación de usuarios	16
Módulo/Paquete	16
Descripción del Dominio	16
Código problemático	16
Solución a Implementar	17
Ventajas	17
Desventajas	17
Propuesta de mejora 5: Falta de specification pattern para búsquedas complejas	18
Descripción del Dominio	18
Código problemático	18
Solución a Implementar	18
Ventajas	18
Desventajas	19
Caso disruptivo	19
Nuevos requerimientos	19
Restricciones técnicas	19
Impacto	19
Solución Propuesta	20
Arquitectura	20
Nivel 1: Diagrama de contexto	20
Nivel 2: Diagrama de contenedores	21
Nivel 3: Diagrama de componentes	22
Frontend:	22
Backend:	23
Entidades Principales	24
Capa de Servicios	24
Capa de Controllers	24
DTOs y Mappers	25
Buenas prácticas	25
Information Expert	25
Creator	26
Low Coupling (Bajo Acoplamiento)	26
Interface Segregation Principle (ISP)	27
Dependency Inversion Principle (DIP)	28
Beneficios en testing	28
Conclusión	28

Descripción del dominio

Misión

Nuestra misión es centralizar y digitalizar la totalidad de la experiencia del centro deportivo. Buscamos trascender la simple gestión administrativa de clases e inscripciones para convertir la plataforma en un compañero de fitness integral para el socio. El objetivo es empoderar a los socios con herramientas para visualizar y alcanzar sus metas de entrenamiento, al mismo tiempo que se dota a instructores y administradores de un potente software de gestión para optimizar la asignación de recursos y el seguimiento de miembros. Adicionalmente, se integra un asistente virtual conversacional impulsado por IA para brindar soporte instantáneo y personalizado a los usuarios.

Alcance

- Módulo de gestión de usuarios (Auth/Admin): Un sistema de autenticación basado en JWT y autorización por roles (Socio, Instructor, Admin) y permisos granulares (PermissionLevel para Admin). Incluye el CRUD completo de usuarios y la autogestión de perfiles.
- Módulo de gestión de actividades: La gestión de inscripciones y bajas de socios a las actividades deportivas, así como la consulta y búsqueda de las mismas.
- Módulo de asistencia virtual (Chatbot): Un chatbot conversacional (ChatBum) que utiliza Google Gemini AI para responder consultas en lenguaje natural sobre actividades, horarios, instructores y disponibilidad. Proporciona información personalizada a usuarios sobre sus inscripciones y estado.

Descripción general

Este proyecto es una solución web integral que moderniza la administración de un centro deportivo. Se compone de un backend (API REST) desarrollado en Java 21 con Spring Boot, que maneja toda la lógica de negocio, la seguridad con JWT y la persistencia de datos en una base de datos MySQL. La arquitectura del backend sigue un diseño en capas (Controller, Service, Repository), con manejo centralizado de excepciones y validación de entradas.

El frontend es una SPA Angular que consume la API, ofreciendo una experiencia de usuario fluida y adaptada a cada rol:

1. Público General: Puede explorar las actividades y registrarse o iniciar sesión. También puede consultar al asistente virtual sobre información general de las actividades disponibles.
2. Socios: Acceden a su perfil, gestionan sus inscripciones a clases. El chatbot les proporciona asistencia personalizada sobre sus inscripciones activas y responde dudas sobre las actividades del centro.

3. Instructores: Además de gestionar sus clases e inscripciones, pueden registrar nuevos socios. Pueden consultar al chatbot información sobre las actividades que dictan y los socios inscriptos.
4. Administradores: Tienen control total sobre la plataforma, gestionando todas las cuentas de usuario, las inscripciones.

El sistema integra un widget de chat flotante disponible en todas las páginas, que mantiene la conversación y accede a información en tiempo real del backend para proporcionar respuestas actualizadas y contextualizadas mediante inteligencia artificial generativa.

Listado de requisitos funcionales

RF-01. El usuario anónimo puede consultar el listado de todas las actividades disponibles.

RF-02. El usuario anónimo puede visualizar el detalle de una actividad seleccionada, incluyendo horario, instructor y precio.

RF-03. El usuario anónimo puede registrarse como socio.

RF-04. El usuario anónimo puede iniciar sesión.

RF-05. El socio puede consultar todas las actividades disponibles.

RF-06. El socio puede ver el detalle de una actividad (nombre, instructor, horario, precio).

RF-07. El socio puede buscar actividades por nombre.

RF-08. El socio puede buscar actividades por rango horario.

RF-09. El socio puede ver el detalle del instructor que dicta una actividad.

RF-10. El socio puede inscribirse a una actividad.

RF-11. El socio puede ver el listado de actividades en las que está inscripto.

RF-12. El socio puede darse de baja de una actividad.

RF-13. El socio puede editar su información personal.

RF-14. El socio puede eliminar su cuenta.

RF-15. El instructor puede registrar nuevos socios.

RF-16. El instructor puede listar a todos los socios.

RF-17. El instructor puede ver el detalle de un socio.

RF-18. El instructor puede ver el listado de todas las actividades del sistema.

RF-19. El instructor puede ver el listado de actividades que dicta.

RF-20. El instructor puede ver el detalle de una actividad.

RF-21. El instructor puede inscribir un socio en una actividad que dicta.

RF-22. El instructor puede dar de baja a un socio de una actividad que dicta.

RF-23. El administrador puede crear socios, instructores y otros administradores.

RF-24. El administrador puede listar todos los usuarios (socios, instructores, administradores).

RF-25. El administrador puede ver el detalle de cualquier usuario y actividad.

RF-26. El administrador puede eliminar socios e instructores.

RF-27. El administrador puede inscribir a un socio en cualquier actividad.

RF-28. El administrador puede dar de baja a un socio de cualquier actividad.

RF-29. El usuario anónimo puede interactuar con el chatbot para consultar información general sobre actividades.

RF-30. El socio puede usar el chatbot para consultar información personalizada sobre sus inscripciones.

RF-31. El chatbot puede proporcionar información sobre horarios, instructores y disponibilidad de cupos de las actividades.

RF-32. El chatbot puede responder preguntas en lenguaje natural sobre las actividades del centro deportivo.

Tecnologías

Backend

Lenguaje: Java 21

Framework: Spring Boot 3.4.5

Base de datos: MySQL (Connector 9.3.0)

ORM: Spring Data JPA

Seguridad: Spring Security con JWT

Documentación: OpenAPI / Swagger (SpringDoc 2.8.9)

Frontend

Lenguaje: TypeScript 5.9.2

Framework: Angular 20.3.0

Build tool: Angular CLI 20.3.3

Librerías:

chart.js

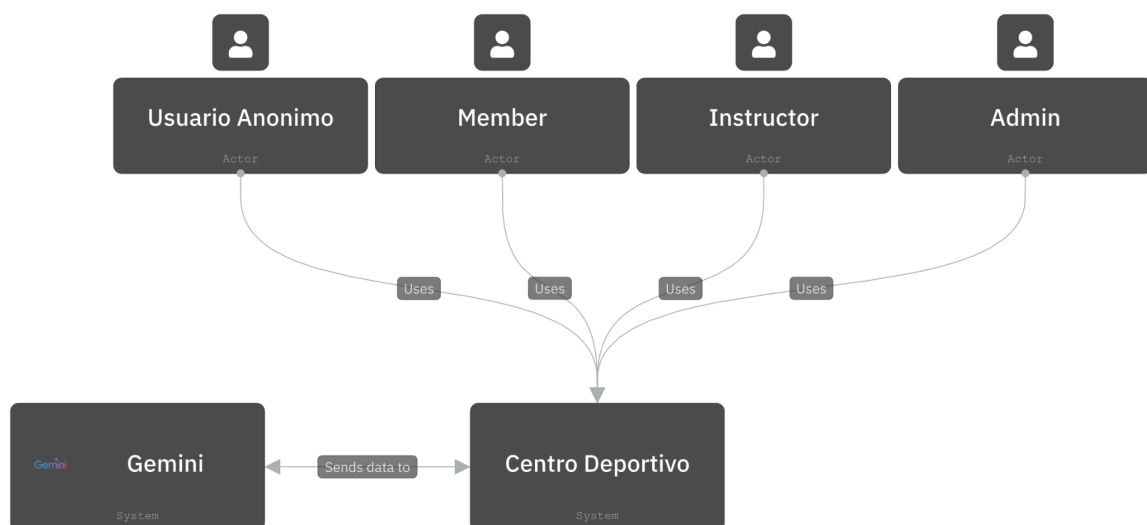
swiper

jwt-decode

rxjs

Detalle de arquitectura

Nivel 1: Diagrama de contexto



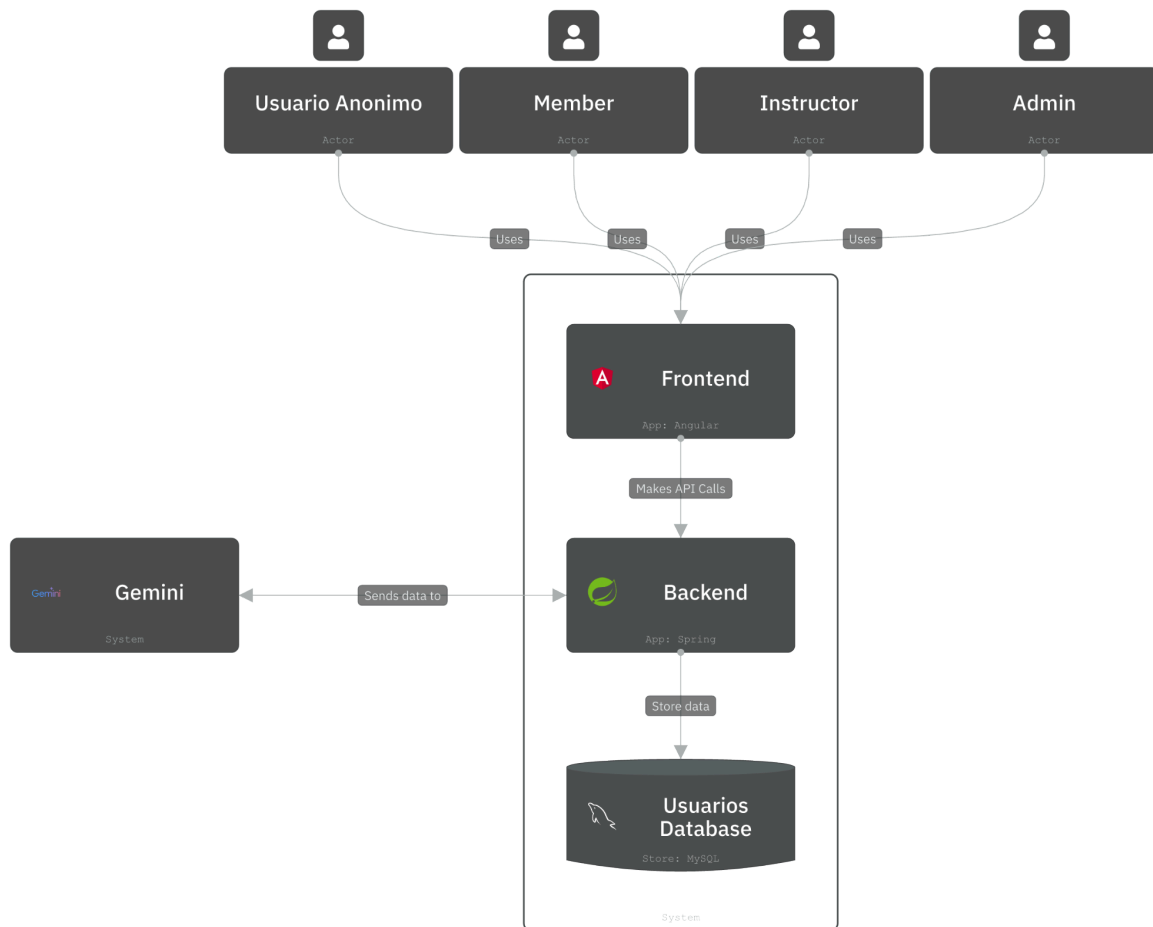
Descripción

El sistema es una aplicación web que permite gestionar actividades deportivas, usuarios e inscripciones en un centro deportivo. Conecta a usuarios anónimos, socios, instructores y administradores para consultar, gestionar e inscribirse en actividades. Integra un chatbot con IA para asistencia informativa.

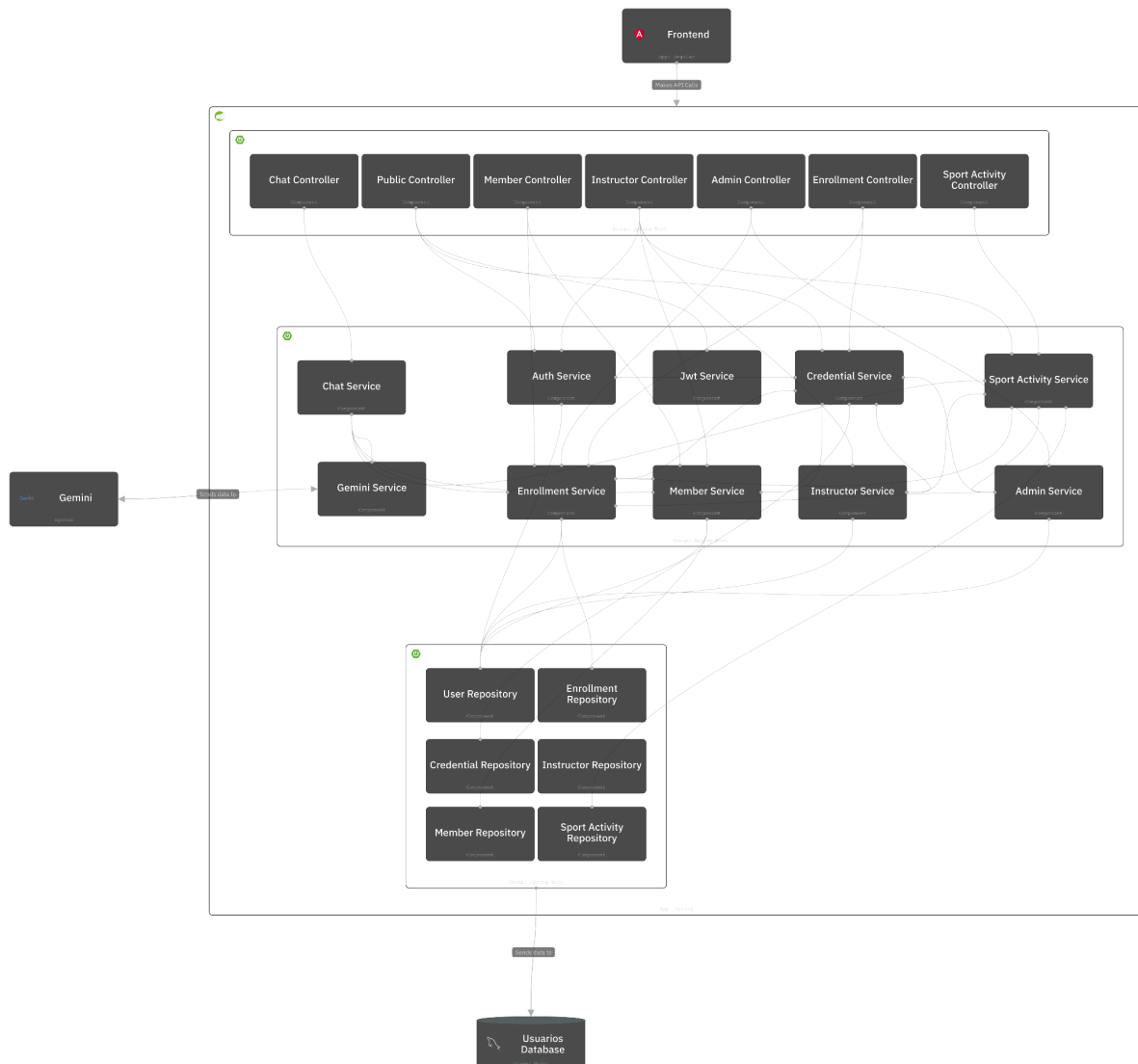
Responsabilidades

- Usuario Anónimo (Actor): Consultar actividades disponibles, ver detalles de actividades e instructores, registrarse como socio, iniciar sesión.
- Member/Socio (Actor): Gestionar su perfil, buscar actividades por nombre o horario, inscribirse y darse de baja de actividades, consultar sus inscripciones.
- Instructor (Actor): Registrar nuevos socios, listar y consultar socios, ver todas las actividades, gestionar inscripciones en sus actividades.
- Administrador (Actor): Crear usuarios (socios, instructores, administradores), gestión total de usuarios, inscribir o dar de baja socios en cualquier actividad, control granular mediante niveles de permiso.
- Centro Deportivo: Proporciona UI web, autentica usuarios con JWT, gestiona usuarios y actividades, controla inscripciones y cupos, almacena datos en MySQL, integra chatbot con IA.
- Gemini API: Provee servicio de chatbot con inteligencia artificial para responder consultas sobre actividades, horarios e inscripciones.

Nivel 2: Diagrama de contenedores



Nivel 3: Diagrama de componentes



Capa de Controllers

Exponer endpoints, validar requests, manejar respuestas HTTP.

- **PublicController:** Login y registro de socios (no autenticado)
- **MemberController:** Gestión de perfil, inscripciones y actividades del socio
- **InstructorController:** Visualización de actividades y gestión de inscripciones de sus clases
- **AdminController:** CRUD completo de usuarios y gestión global de inscripciones
- **SportActivityController:** Consulta de actividades deportivas
- **ChatController:** Procesamiento de mensajes del chatbot, endpoint de salud del servicio

Capa de Services

Lógica de negocio, validaciones.

- **AuthService:** Registro de usuarios, validación de campos únicos, encriptación

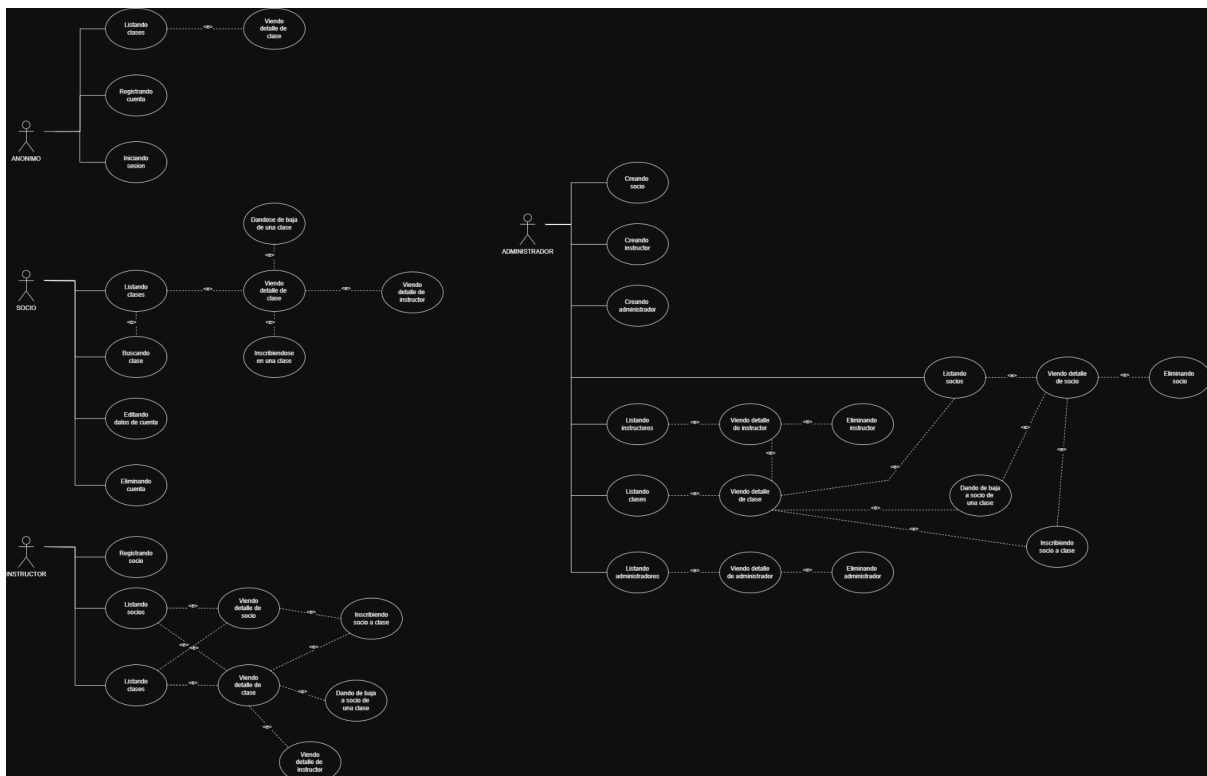
- MemberService: Gestión de perfil de socios, cambio de estado
- InstructorService: Gestión de instructores y consulta de sus actividades
- AdminService: Creación y eliminación de usuarios según permisos
- SportActivityService: Búsqueda y filtrado de actividades
- EnrollmentService: Lógica de inscripción, validación de cupos, gestión de estados
- CredentialService: Autenticación, carga de UserDetails, validación de usuarios
- JwtService: Generación y validación de tokens JWT
- ChatService: Orquestación del chatbot, construcción de contexto del sistema, integración con servicios del negocio
- GeminiService: Comunicación con API externa de Gemini

Capa de Repository

Manejo y persistencia de datos.

- IUserRepository: Operaciones CRUD sobre usuarios, queries por email/DNI
- IMemberRepository, IInstructorRepository: Especialización de queries por tipo de usuario
- ISportActivityRepository: Búsquedas por nombre, rango horario, instructor
- IEnrollmentRepository: Queries de inscripciones por socio/actividad
- ICredentialRepository: Búsqueda por username para autenticación

Diagrama de casos de uso



Estado actual

PROPUESTA DE MEJORA 1: Violación del principio de responsabilidad unica (SRP) en EnrollmentService

Módulo/Paquete

com.utn.API_CentroDeportivo.service.impl.EnrollmentService

Descripción del Dominio

El servicio de inscripciones gestiona la relación entre socios y actividades deportivas. Actualmente, además de manejar la lógica de inscripción/baja, también tiene la responsabilidad de modificar el estado del socio (Status.ACTIVE/INACTIVE), lo cual es una responsabilidad del MemberService.

Código problemático

```
@Transactional 4 usages  Matias Benedetti
public void enrollMemberToActivity(String username, Long activityId) {

    Member member = (Member) credentialService.getUserByUsername(username);
    SportActivity activity = sportActivityService.getSportActivityEntityById(activityId).get();

    if (enrollmentRepository.findByMemberIdAndActivityId(member.getId(), activityId).isPresent()) {
        throw new MemberAlreadyEnrolledException("El socio ya está inscripto en esta actividad");
    }

    Enrollment enrollment = Enrollment.builder() EnrollmentBuilder<capture of ?, capture of ?>
        .startDate(LocalDate.now()) capture of ?
        .endDate(LocalDate.now().plusDays( daysToAdd: 30))
        .member(member)
        .activity(activity)
        .build();

    enrollmentRepository.save(enrollment);
    memberService.updateMemberStatus(member.getId());
}

@Transactional 4 usages  TomasB +1
@Override
public void unsubscribeMemberFromActivity(String username, Long activityId) {
    Member member = (Member) credentialService.getUserByUsername(username);

    Enrollment enrollment = enrollmentRepository
        .findByMemberIdAndActivityId(member.getId(), activityId)
        .orElseThrow(() -> new EnrollmentNotFoundException("Inscripción no encontrada"));

    enrollmentRepository.delete(enrollment);

    boolean hasOtherEnrollments = enrollmentRepository.existsByMemberId(member.getId());
    if (!hasOtherEnrollments) {
        member.setStatus(Status.INACTIVE);
        userRepository.save(member);
    }
}
```

Solución a implementar

Implementar el patrón Observer utilizando el sistema de eventos de Spring para desacoplar completamente la gestión de inscripciones de la actualización del estado de los socios. El `EnrollmentService` publicará eventos de dominio cuando ocurran inscripciones o bajas, y el `MemberService` reaccionará a estos eventos de forma independiente mediante listeners. Esto permite que cada servicio mantenga su responsabilidad única y facilita la extensión futura del sistema con nuevos listeners sin modificar el código existente.

Ventajas

- SRP: Cada servicio tiene una única responsabilidad clara
- Desacoplamiento: `EnrollmentService` no conoce la lógica interna de `Member`
- Escalabilidad: Fácil agregar nuevos listeners (ej: enviar email de bienvenida)
- Testabilidad: Se pueden testear los eventos independientemente
- Trazabilidad: Los eventos documentan acciones importantes del sistema

Desventajas

- Complejidad inicial: Requiere entender el sistema de eventos de Spring
- Debugging: El flujo es menos directo (requiere seguir eventos)
- Performance: Overhead mínimo por el dispatcher de eventos

Propuesta de mejora 2: Duplicación de código

Módulo/Paquete

`com.utn.API_CentroDeportivo.model.mapper` - `MemberMapper`, `InstructorMapper`, `AdminMapper`

Descripción del Dominio

Los mappers convierten entidades en DTOs y viceversa. Actualmente, cada mapper tiene código repetido para mapear los campos comunes de `User` (name, lastname, dni, email, etc.).

Código problemático

```
public class MemberMapper {  
  
    public static Member mapToMember(MemberRequestDTO memberDTO) {  
        Member member = new Member();  
        member.setName(memberDTO.getName());  
        member.setLastname(memberDTO.getLastname());  
        member.setDni(memberDTO.getDni());  
        member.setBirthdate(memberDTO.getBirthdate());  
        member.setPhone(memberDTO.getPhone());  
        member.setEmail(memberDTO.getEmail());  
        return member;  
    }  
}
```

```
public class InstructorMapper {  
  
    public static Instructor mapToInstructor(UserRequestDTO dto) {  
        Instructor instructor = new Instructor();  
        instructor.setName(dto.getName());  
        instructor.setLastname(dto.getLastname());  
        instructor.setDni(dto.getDni());  
        instructor.setBirthdate(dto.getBirthdate());  
        instructor.setPhone(dto.getPhone());  
        instructor.setEmail(dto.getEmail());  
        instructor.setSpecialty(dto.getSpecialty());  
        return instructor;  
    }  
}
```

```
public class AdminMapper {  
  
    public static Admin mapToAdmin(UserRequestDTO dto) {  
        Admin admin = new Admin();  
        admin.setName(dto.getName());  
        admin.setLastname(dto.getLastname());  
        admin.setDni(dto.getDni());  
        admin.setBirthdate(dto.getBirthdate());  
        admin.setPhone(dto.getPhone());  
        admin.setEmail(dto.getEmail());  
        admin.setPermissionLevel(dto.getPermissionLevel());  
        admin.setHireDate(LocalDate.now());  
        return admin;  
    }  
}
```

Solución a implementar

Aplicar el patrón Template Method creando una clase base abstracta que centralice el mapeo de campos comunes de User. La clase base definirá el esqueleto del proceso de mapeo en un método plantilla que incluye la lógica compartida, mientras que los mappers

específicos implementarán métodos abstractos para la creación de entidades concretas y el mapeo de campos particulares. Esto elimina la duplicación de código, garantiza consistencia en el proceso de mapeo y facilita el mantenimiento cuando se agregan o modifican campos comunes.

Ventajas

- DRY: Código común en un solo lugar.
- Mantenibilidad: Cambios en User se hacen una vez.
- Consistencia: Todos los mappers siguen la misma estructura.
- Extensibilidad: Fácil agregar nuevos tipos de usuarios.

Desventajas

- Herencia vs Composición: Viola "Favor Composition over Inheritance" (mitigable con interfaces).
- Complejidad: Requiere entender la jerarquía para modificar.
- Tolerante a cambios: Muy alta, nuevos campos comunes solo afectan la clase base.
- Viabilidad: Alta, pero requiere refactoring de todos los mappers existentes.

Propuesta de mejora 3: Alto acoplamiento en MemberService

Módulo/Paquete

`com.utn.API_CentroDeportivo.service.impl.MemberService`

Descripción del Dominio

MemberService depende directamente de implementaciones concretas y tiene demasiadas dependencias inyectadas. Además, mezcla lógica de validación con lógica de negocio.

Código problemático

```
@Service
public class MemberService implements IMemberService {

    @Autowired
    private IUserRepository userRepository;

    @Autowired
    private IMemberRepository memberRepository;

    @Autowired
    private ICredentialService credentialService;

    @Transactional
    public void updateMemberStatus(Long memberId) {...}

    @Override
    public Optional<Member> getMemberById(Long memberId) {...}

    @Transactional
    @Override
    public void updateMemberProfile(String username, MemberEditDTO dto) {
        Member member = (Member) userRepository.findById(credentialService.getUserByUsername(username).getId())
            .orElseThrow(() -> new MemberNotFoundException("Socio no encontrado"));

        if (dto.getName() != null) {
            member.setName(dto.getName());
        }
        if (dto.getLastname() != null) {
            member.setLastname(dto.getLastname());
        }
        if (dto.getPhone() != null) {
            member.setPhone(dto.getPhone());
        }
        if (dto.getEmail() != null) {
            if (!dto.getEmail().equals(member.getEmail()) && userRepository.existsByEmail(dto.getEmail())) {
                throw new FieldAlreadyExistsException("email", "El campo ya está registrado");
            }
        }
    }
}
```

Solución a Implementar

Simplificar el servicio de socios mediante dos estrategias complementarias. Primero, crear una clase "fachada" que contenga toda la complejidad de obtener un usuario. Actualmente, para obtener un socio hay que primero buscar sus credenciales, luego obtener el ID, y finalmente buscar el socio por ese ID. La fachada encapsula todo eso en un solo método simple. Segundo, separar todas las validaciones (como verificar si un email ya existe) en componentes independientes que se pueden intercambiar fácilmente. El servicio principal queda mucho más limpio: solo obtiene el usuario, ejecuta las validaciones necesarias, y guarda los cambios. Esto hace que el código sea más fácil de entender, probar y mantener, además de reducir la cantidad de dependencias directas que tiene el servicio.

Ventajas

- SRP: Validación separada de lógica de negocio
- Testabilidad: Se pueden mockear estrategias fácilmente
- Reutilización: Estrategias de validación compartibles
- Clean Code: Métodos más cortos y legibles

Desventajas

- Más clases: Aumenta el número de archivos del proyecto
- Curva de aprendizaje: Desarrolladores nuevos deben entender los patrones
- Tolerancia a cambios: Muy alta, nuevas validaciones se agregan como estrategias
- Viabilidad: Alta, refactoring incremental posible

Propuesta de mejora 4: Patrón factory para creación de usuarios

Módulo/Paquete

com.utn.API_CentroDeportivo.service.impl - AuthService, AdminService

Descripción del Dominio

La creación de usuarios (Member, Instructor, Admin) tiene lógica duplicada y no centralizada. Cada servicio tiene su propia forma de crear usuarios, encriptar passwords y asignar roles.

Código problemático

AuthService:

```
@Transactional
public void registerMember(MemberRequestDTO memberDTO) {
    Member member = MemberMapper.mapToMember(memberDTO);
    member.setStatus(Status.INACTIVE);
    Credential credential = Credential.builder().username(memberDTO.getUsername()).password(memberDTO.getPassword()).build();
    member.setCredential(credential);
    createAndSaveUser(member, Role.MEMBER);
}
```

AdminService:



```
@Override
public void createUser(UserRequestDTO userDTO) {
    validate(userDTO);

    User user;
    switch (userDTO.getRole()) {
        case MEMBER:
            Member member = MemberMapper.mapToMember(userDTO);
            member.setStatus(Status.INACTIVE);
            user = member;
            break;
        case INSTRUCTOR:
            user = InstructorMapper.mapToInstructor(userDTO);
            break;
        case ADMIN:
            user = AdminMapper.mapToAdmin(userDTO);
            break;
        default:
            throw new InvalidRoleException("El rol especificado no es válido: " + userDTO.getRole());
    }

    Credential credential = Credential.builder()
        .username(userDTO.getUsername())
        .password(userDTO.getPassword())
        .build();
    user.setCredential(credential);

    authService.createAndSaveUser(user, userDTO.getRole());
}
```

Solución a Implementar

Implementar el patrón Abstract Factory para centralizar y estandarizar el proceso de creación de usuarios. Se definirá una interfaz común que declare los métodos necesarios para crear usuarios, credenciales y ejecutar validaciones, con implementaciones concretas para cada tipo de usuario que encapsulen toda la lógica específica de construcción, incluyendo la encriptación de contraseñas y asignación de roles. Los servicios delegarán la creación de usuarios a estas factories, eliminando la duplicación de código y facilitando la incorporación de nuevos tipos de usuarios sin modificar el código existente.

Ventajas

- SRP: Creación de usuarios centralizada y especializada
- Open/Closed: Nuevos tipos de usuarios sin modificar código existente
- Testabilidad: Factories se pueden testear independientemente
- Consistencia: Todos los usuarios se crean de la misma manera estructural

Desventajas

- Complejidad: Más clases e interfaces
- Overhead: Para proyectos pequeños puede ser over-engineering
- Tolerancia a cambios: Muy alta, nuevos tipos de usuarios solo requieren nueva factory

- Viabilidad: Media-Alta, requiere refactoring significativo pero sin breaking changes

Propuesta de mejora 5: Falta de specification pattern para búsquedas complejas

Descripción del Dominio

El patrón Specification resuelve la limitación de búsquedas rígidas al separar la lógica de consulta del servicio y del repositorio, permitiendo construir queries de forma dinámica y combinable en tiempo de ejecución.

Código problemático

```
@Override
public Page<SportActivitySummaryDTO> findActivitiesByName(String name, Pageable pageable) {
    return sportActivityRepository.findByNameContainingIgnoreCase(name, pageable)
        .map(SportActivityMapper::mapToSportActivitySummaryDTO);
}

@Override
public Page<SportActivitySummaryDTO> findActivitiesByTimeRange(String startTime, String endTime, Pageable pageable) {
    try {
        LocalDateTime startTimeFrom = LocalDateTime.parse(startTime);
        LocalDateTime endTimeTo = LocalDateTime.parse(endTime);

        return sportActivityRepository.findByTimeRangeOverlap(startTimeFrom, endTimeTo, pageable)
            .map(SportActivityMapper::mapToSportActivitySummaryDTO);
    } catch (DateTimeParseException e) {
        throw new InvalidTimeFormatException("El formato de hora es inválido.");
    }
}
```

Solución a Implementar

Crear un sistema de filtros modulares que funciona como piezas que se pueden combinar libremente. Cada filtro es independiente y representa un criterio de búsqueda específico. Estos filtros se pueden combinar usando operadores lógicos para crear búsquedas complejas según lo que el usuario necesite en cada momento. Si los criterios cambian, simplemente se combinan otros filtros. La incorporación de nuevos criterios de búsqueda se realiza mediante la creación de filtros adicionales, sin necesidad de modificar la lógica existente. Esto hace que el sistema de búsqueda sea extremadamente flexible y fácil de extender.

Ventajas

- Composición dinámica: Combinar filtros fácilmente
- Reutilización: Specifications compartibles en diferentes servicios
- Extensibilidad: Agregar nuevos filtros sin modificar existentes (Open/Closed)
- Type-safe: Compilador detecta errores en queries
- Testabilidad: Cada specification se testea independientemente

Desventajas

- Curva de aprendizaje: API de Criterios de JPA es compleja
- Performance: Queries dinámicas pueden ser menos optimizadas que JPQL hardcodeado
- Debugging: Queries generadas dinámicamente son más difíciles de visualizar
- Tolerancia a cambios: Muy alta, sistema de filtros completamente extensible
- Viabilidad: Alta, compatible con código existente (métodos legacy se pueden mantener)

Caso disruptivo

El centro deportivo desea expandir su oferta de servicios implementando un sistema completo de rutinas de entrenamiento personalizadas. Este nuevo módulo permitirá a instructores y socios gestionar planes de entrenamiento estructurados, hacer seguimiento del progreso y visualizar la evolución en el tiempo.

Nuevos requerimientos

Para instructores:

- Crear rutinas de entrenamiento
- Asignar rutinas personalizadas a los socios
- Modificar las rutinas creadas por el propio instructor

Para socios:

- Crear rutinas de entrenamiento
- Visualizar rutinas asignadas por un instructor
- Registrar entrenamientos realizados
- Consultar gráfico de progreso por ejercicio

Restricciones técnicas

Los datos de rutinas deben almacenarse en una base de datos MySQL independiente de la base de datos principal.

El sistema debe usar Spring Boot como framework.

La relación entre usuarios y rutinas debe manejarse mediante el username.

Impacto

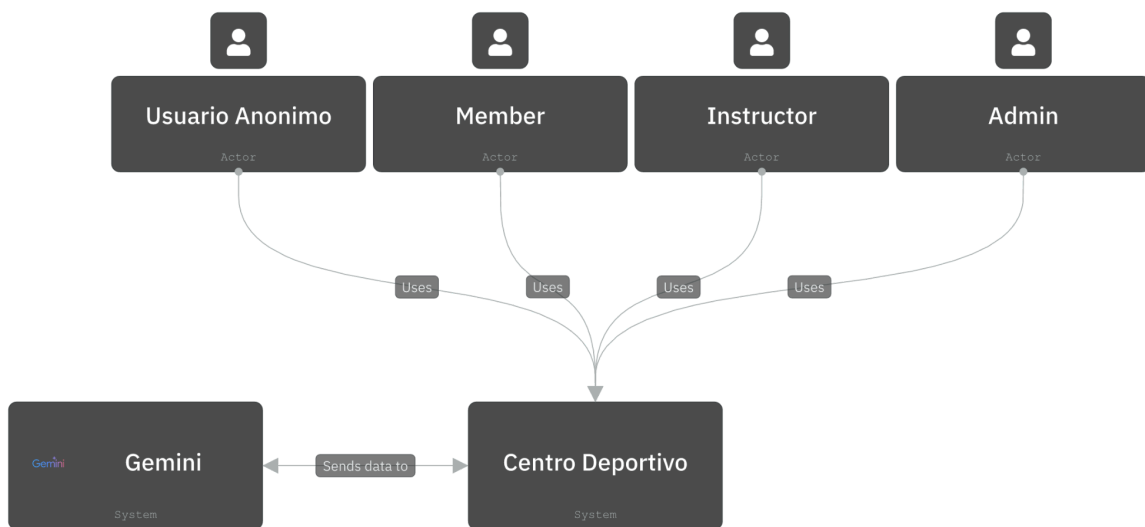
Capa	Archivos	Complejidad
Entidades	~5	Media
Repositorios	3 - 5	Media
Servicios	5 - 10	Alta
Controllers	3 - 4	Media

DTOs	~12	Baja
Excepciones	5 - 10	Baja
Tests		Alta

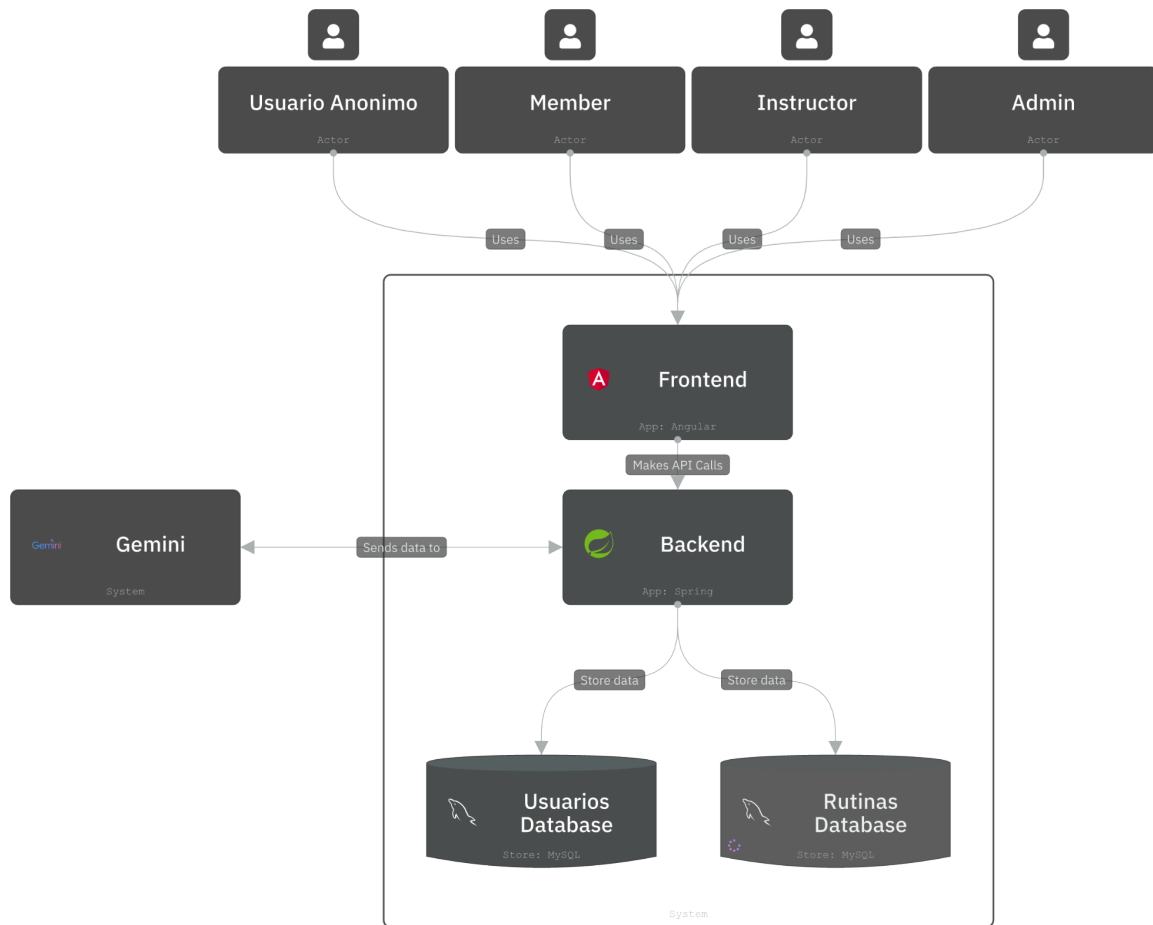
Solución Propuesta

Arquitectura

Nivel 1: Diagrama de contexto

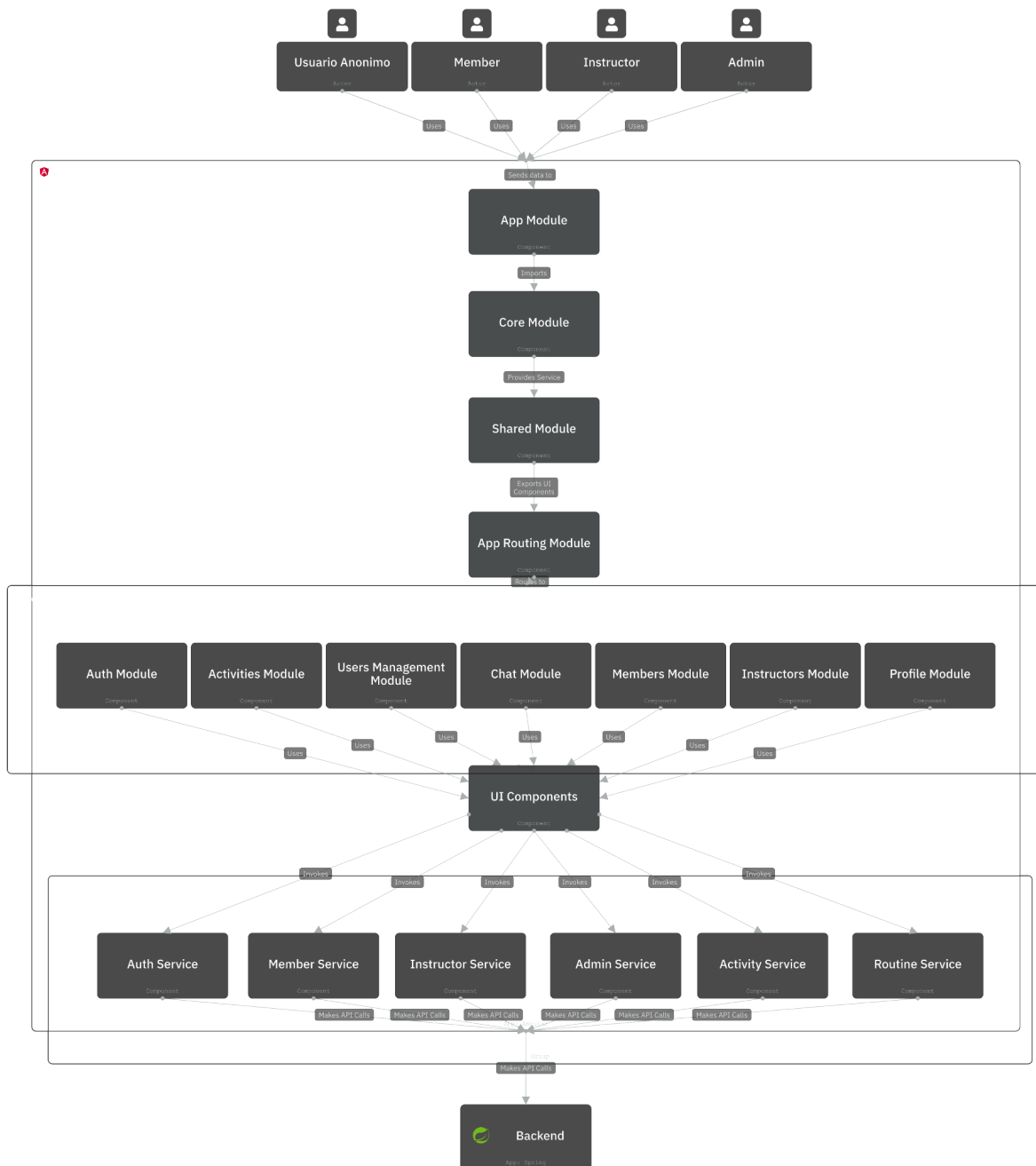


Nivel 2: Diagrama de contenedores

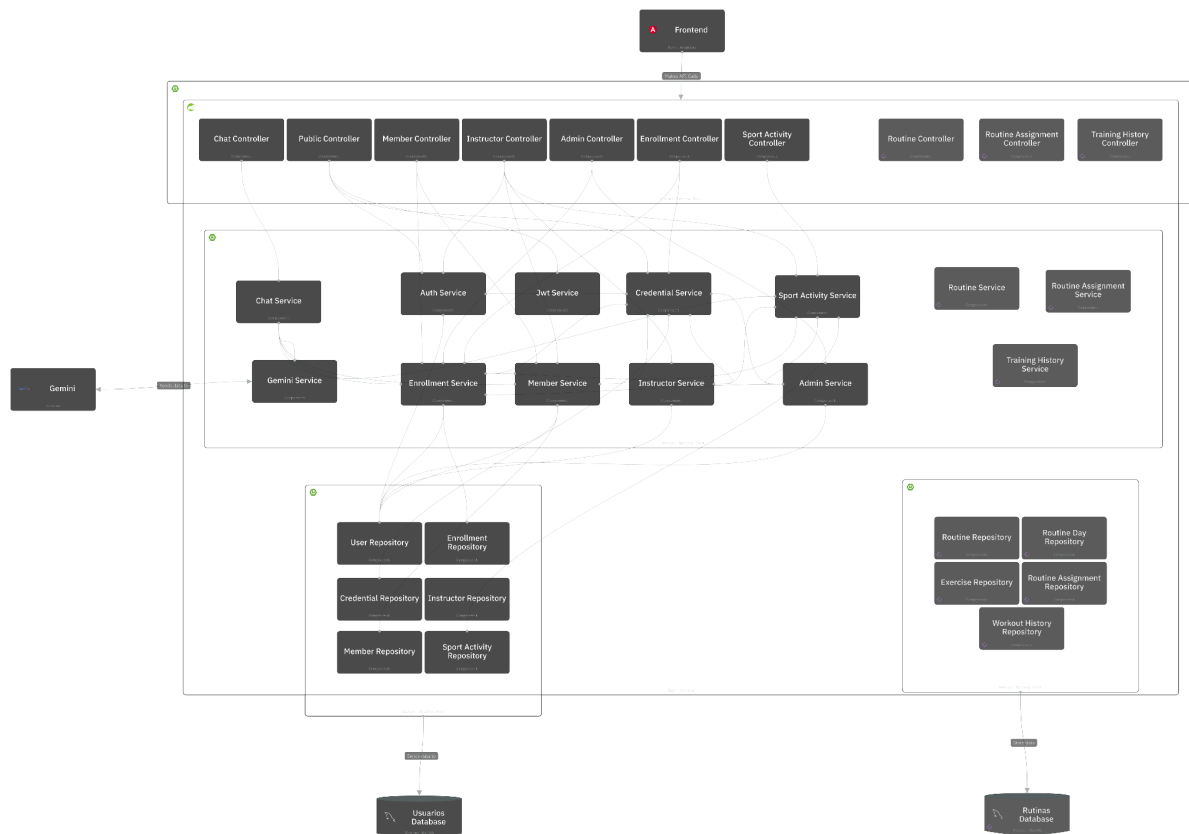


Nivel 3: Diagrama de componentes

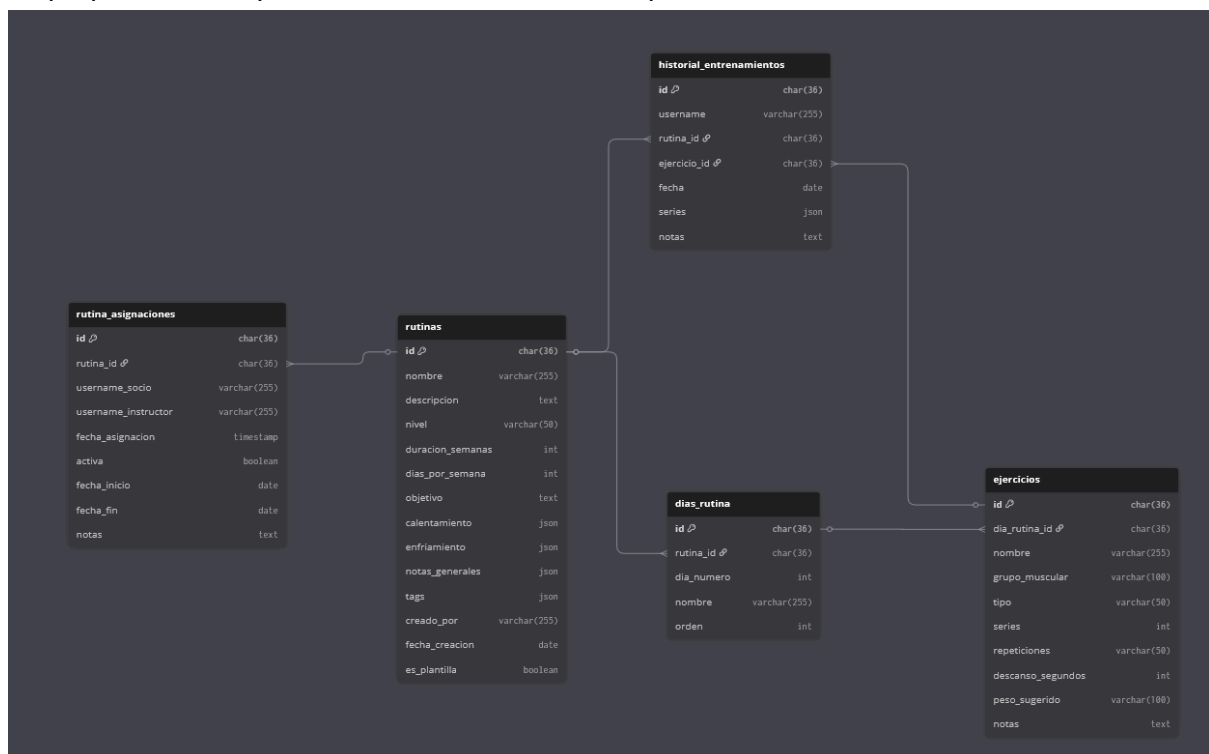
Frontend:



Backend:



Se propone una arquitectura de base de datos separada:



Entidades Principales

El sistema se estructura en torno a cinco entidades principales, que representan los distintos aspectos del dominio de entrenamiento:

Routine: Contiene la información general de cada rutina, como nombre, descripción, nivel, duración y objetivo. Además, almacena el username del creador y su tipo (SOCIO o INSTRUCTOR). Posee relaciones OneToMany con RoutineDay(días que componen la rutina) y con RoutineAssignment(usuarios a los que fue asignada).

RoutineDay: Representa cada día de entrenamiento dentro de una rutina. Incluye datos como número, nombre y orden. Mantiene una relación ManyToOne con Routine y una OneToMany con Exercise, agrupando los ejercicios correspondientes a ese día.

Exercise: Define los detalles técnicos de cada ejercicio, como nombre, grupo muscular, tipo, número de series, repeticiones, descanso, peso sugerido y notas. Se relaciona ManyToOne con RoutineDay, lo que permite organizar los ejercicios dentro de cada jornada de entrenamiento.

RoutineAssignment: Vincula rutinas con usuarios, guardando los usernames del socio y del asignador, las fechas de asignación y finalización, el estado activo y las notas correspondientes. Su relación principal es ManyToOne con Routine.

TrainingHistory: Registra la ejecución de los entrenamientos realizados por los usuarios. Contiene el username del usuario, la fecha del entrenamiento y un campo JSON con la información detallada de las series, pesos y repeticiones. Se relaciona ManyToOne con Exercise, permitiendo asociar cada registro con el ejercicio específico realizado.

Capa de Servicios

La lógica de negocio se distribuye en tres servicios principales

RoutineService: Gestiona las operaciones CRUD de rutinas, incluyendo la validación de propiedad (ownership) y la verificación de existencia de usuarios en la base de datos principal.

RoutineAssignmentService: Se encarga de la asignación y desasignación de rutinas, validando los permisos de los instructores, controlando los estados activos y garantizando las reglas de negocio relacionadas con las rutinas asignadas.

TrainingHistoryService: Administra el registro de entrenamientos, valida que los ejercicios pertenezcan a rutinas asignadas al usuario, y estructura correctamente los datos JSON que representan las series realizadas.

Capa de Controllers

La capa de presentación se organiza en tres controllers principales

RoutineController: Implementa el CRUD completo de rutinas, junto con funcionalidades adicionales como listados paginados y consulta de plantillas públicas.

RoutineAssignmentController: Gestiona las operaciones de asignación realizadas por instructores, la consulta de rutinas asignadas a socios y la desactivación de asignaciones activas.

TrainingHistoryController: Permite registrar entrenamientos y consultar el historial por ejercicio o por usuario, facilitando el seguimiento del progreso.

DTOs y Mappers

DTOs Request: Diseñados para la recepción de datos.

DTOs Response: Divididos en diferentes niveles de detalle: Summary DTOs para listados y Detail DTOs para vistas completas con estructura anidada.

Mappers: Implementan la conversión bidireccional entre entidades y DTOs, gestionando correctamente estructuras complejas y campos en formato JSON.

Buenas prácticas

Information Expert

En nuestra arquitectura, diseñamos cada servicio como el experto de su dominio específico.

RoutineService es el experto en rutinas. Conoce su estructura completa, sus validaciones y quién es el creador. También decide si una rutina puede eliminarse, porque sabe si tiene asignaciones activas, el historial asociado y quien es el propietario.

De la misma manera, **AssignmentService** es el experto en asignaciones. Maneja toda la información sobre qué rutina está asignada a qué usuario, quién la asignó y su estado. Por eso es el encargado de validar si un instructor puede asignar cierta rutina a un socio.

ExerciseService es el experto en ejercicios individuales: conoce a qué día pertenecen, características técnicas (series, repeticiones), y puede validar si un ejercicio puede ser modificado o eliminado según su contexto dentro de la rutina.

Esta distribución de responsabilidades reduce el acoplamiento, aumenta la cohesión y evita que los servicios tengan que consultarse entre sí para tomar decisiones.

Creator

Este principio define quién debe ser responsable de crear objetos. En nuestro proyecto, dejamos esa responsabilidad en los servicios, la clase que agrega, contiene, registra o usa el objeto creado, ya que conocen las reglas de negocio y la información necesaria para crear cada entidad correctamente.

RoutineService crea las instancias de Routine porque necesita obtener el usuario autenticado, verificar su rol, validar que exista en la base de datos, asignar valores por defecto y dejar la rutina en un estado inicial válido.

AssignmentService crea instancias de RoutineAssignment porque debe verificar la existencia de la rutina y del usuario, validar quién realizó la asignación (instructor o auto-asignación del socio), determinar fechas de inicio y fin según las políticas del negocio, y desactivar asignaciones previas si es necesario según la regla de una sola rutina activa por usuario.

HistoryService crea TrainingHistory porque tiene que validar que el ejercicio pertenezca a una rutina asignada, estructurar el JSON de series, asociarlo al usuario autenticado y registrar la fecha del entrenamiento.

Esta centralización de la creación tiene varias ventajas: si cambian las reglas de inicialización, solo modificamos el servicio creador. Además, evitamos duplicación de lógica de validación.

Low Coupling (Bajo Acoplamiento)

Aplicamos el principio de bajo acoplamiento para reducir al mínimo las dependencias entre módulos, de modo que los cambios en uno afecten mínimamente a otros. Para lograrlo, separamos el sistema en dos bases de datos independientes: una para rutinas y otra para usuarios/actividades.

Diseñamos el módulo de rutinas y el módulo de usuarios/actividades como completamente desacoplados a nivel de persistencia. No usamos foreign keys reales entre módulos; solo nos comunicamos mediante el username como String. Esto permite que:

- El módulo de rutinas pueda cambiar sin afectar al de usuarios (nuevas tablas, cambios de estructura, incluso cambiar de motor de base de datos).
- El módulo de usuarios también pueda evolucionar sin impactar al de rutinas.

El username funciona como un contrato mínimo entre ambos, manteniendo las dependencias simples y claras.

Gracias a este bajo acoplamiento, en un futuro sería fácil pasar a una arquitectura de microservicios: simplemente separar el módulo de rutinas y utilizarlo como un servicio independiente. Además, si una base de datos falla, el otro módulo puede seguir funcionando sin problemas.

Interface Segregation Principle (ISP)

Aplicamos este principio evitando crear interfaces muy grandes. En vez de eso, usamos interfaces pequeñas y específicas, para que cada parte del sistema use solo los métodos que realmente necesita.

IRoutineService: Tendrá únicamente métodos de gestión de rutinas, como: createRoutine(), updateRoutine(), deleteRoutine(), getRoutines(), getRoutine().

IAssignmentService: Se ocupará solo de las asignaciones:
`assignRoutine()`, `cancelAssignment()`.

IHistoryService: Maneja exclusivamente el historial de entrenamientos:
`registerTraining()`, `getHistoryByExercise()`, `getHistoryByUser()`, `deleteRecord()`.

Esta separación hace que el código sea más claro y fácil de mantener. Cada controller usa solo la interfaz que necesita, el código queda más ordenado y los tests son más fáciles de armar porque solo se mockea la interfaz correspondiente.

Dependency Inversion Principle (DIP)

En nuestro proyecto aplicamos el principio de Inversión de Dependencias haciendo que las capas no dependan de implementaciones concretas, sino de interfaces.

Capa de Controllers → Services:

Los controllers usan interfaces como `IRoutineService` en lugar de la clase concreta. Así, si en algún momento cambiamos la implementación del servicio, el controller no necesita modificarse.

Capa de Services → Repositories:

Los servicios dependen de interfaces de repositorio y no de la implementación que usa Spring Data JPA. Esto permite cambiar la forma de acceder a los datos (otro ORM, una API, etc.) sin alterar el servicio.

De esta forma mantenemos el código más flexible y fácil de mantener.

Beneficios en testing

En nuestros tests unitarios de `RoutineService`, no necesitamos levantar una base de datos real ni usar un `EntityManager`. Podemos crear mocks de `IRoutineRepository` que simulan comportamientos. Esto hace nuestros tests rápidos y enfocados únicamente en la lógica de negocio del servicio.

Conclusión

La arquitectura que se propone organiza el sistema de rutinas en componentes especializados que manejan la complejidad dividiéndola en partes manejables: servicios enfocados en rutinas, asignaciones e historial de entrenamientos. Al separar las bases de datos, el módulo funciona de forma independiente sin afectar al sistema actual, lo que facilita el mantenimiento y reduce riesgos. El diseño está preparado para agregar nuevas funcionalidades sin modificar código existente gracias a los principios de bajo acoplamiento e interfaces específicas. No requiere cambios en las responsabilidades actuales del sistema, ya que se conecta únicamente mediante el username del usuario. El esfuerzo de implementación es moderado por la cantidad de componentes nuevos, pero no introduce tecnologías desconocidas para el grupo. El mayor esfuerzo estará en el desarrollo y testing, especialmente en validaciones de permisos y la integridad del historial de entrenamientos.



UTNMDP
Regional Mar del Plata

Centro Deportivo